Rapport de Projet

Traitement & Analyse de Données

Détection de Ransomwares

Preprocessing, Analyse Discriminante & Génération de Données Synthétiques

Encadré par :

Pr. Hamza Kamal Idrissi

Réalisé par :

Ahmed Reda Sahabi

Youssef Figuigui

Ilyas Bejja

Année Universitaire **2025 — 2026**

Table des matières

1 Introduction **1**

1.1 Contexte et problématique 1

1.2 Objectifs du travail 1

1.3 Description du dataset 2

1.4 Pipeline global du projet 2

2 Exploration Initiale du Dataset **3**

2.1 Aperçu général (dimensions et types de données) 3

2.2 Aperçu des premières lignes 3

2.3 Statistiques descriptives et distribution des classes 4

2.3.1 Distribution des classes 4

3 Prétraitement (Preprocessing) **5**

3.1 Analyse des valeurs manquantes 5

3.2 Détection et suppression des doublons 5

3.3 Traitement des valeurs aberrantes (Outliers) 6

3.3.1 Détection des outliers — méthode IQR $\times$ 1.5 6

3.3.2 Application du capping conservateur — IQR $\times$ 3 6

3.4 Encodage de la variable cible 8

4 Feature Engineering **9**

4.1 Introduction 9

4.2 Agrégations par groupe fonctionnel (+14 features) 9

4.3 Ratios comportementaux (+7 features) 10

4.4 Indicateurs binaires de risque (+6 features) 10

4.5 Diversité et sparsité (+5 features) 11

4.6 Résumé des features ajoutées 11

5 Sélection des features **13**

5.1 Introduction . . . 13
5.2 Analyse de corrélation . . . 13
5.3 Test de Mann-Whitney U . . . 14
5.4 Information mutuelle . . . 15
5.5 Importance Random Forest . . . 16
5.6 Synthèse et features retenues . . . 17

6 Transformation des données . . . 18
6.1 Introduction . . . 18
6.2 Transformation logarithmique (log1p) . . . 18
6.3 Normalisation z-score (StandardScaler) . . . 19
6.4 Synthèse du pipeline de transformation . . . 20

7 Récapitulatif et perspectives . . . 21
7.1 Ce qui a été réalisé . . . 21
7.2 Ce qui reste à faire . . . 21

8 Augmentation par TGAN . . . 22
8.1 Introduction . . . 22
8.2 Implémentation . . . 22
8.3 Évaluation des classifieurs . . . 23
8.4 Synthèse et analyse . . . 24

9 Augmentation par CTGAN . . . 25
9.1 Introduction . . . 25
9.2 Implémentation . . . 25
9.3 Évaluation des classifieurs . . . 26
9.4 Synthèse et analyse . . . 26

10 Augmentation par TVAE . . . 28
10.1 Introduction . . . 28
10.2 Implémentation . . . 28
10.3 Évaluation des classifieurs . . . 28
10.4 Synthèse et analyse . . . 29

Conclusion générale . . . 31

Ressources et Code Source . . . 34

1. Introduction

1.1 Contexte et problématique

La prolifération des cyberattaques de type *ransomware* représente aujourd'hui l'une des menaces les plus critiques en matière de sécurité informatique. Ces logiciels malveillants chiffrent les fichiers des victimes et exigent le paiement d'une rançon en échange de leur restitution, causant des pertes économiques et opérationnelles considérables à l'échelle mondiale.

Face à l'évolution permanente de ces menaces, les méthodes de détection traditionnelles basées sur les signatures se révèlent insuffisantes et incapables d'identifier des variantes inconnues ou obfusquées. L'apprentissage automatique (*Machine Learning*), en s'appuyant sur les comportements observables lors de l'exécution des binaires — notamment les appels API Windows — s'impose comme une alternative robuste, capable de généraliser à de nouvelles variantes.

Ce travail s'inscrit dans cette démarche en proposant un pipeline complet, allant de la préparation des données jusqu'à l'évaluation comparative de modèles ML et DL sur des données réelles et synthétiques.

1.2 Objectifs du travail

Ce projet s'articule autour de deux axes principaux et complémentaires.

Le **premier axe** porte sur la préparation et l'analyse du dataset : nettoyage des données brutes, construction de nouvelles features comportementales par ingénierie, puis identification des variables les plus discriminantes à travers la Différence Absolue des Moyennes (DAM), le test statistique de Mann-Whitney et l'Analyse en Composantes Principales (ACP).

Le **second axe** concerne l'augmentation des données et l'évaluation des modèles. Plusieurs algorithmes de génération de données synthétiques sont mis en œuvre — TGAN, CTGAN, TVAE et TabSyn — afin de produire des datasets augmentés. Ces derniers, ainsi que le dataset original, sont ensuite évalués et comparés à l'aide d'algorithmes de Machine

Learning et de Deep Learning : Decision Tree, Régression Logistique, SVM, Naïve Bayes, KNN et CNN.

1.3 Description du dataset

Le dataset utilisé regroupe **806 traces d'exécution** de binaires, collectées en environnement sandbox contrôlé et étiquetées en deux classes distinctes : *Goodware* (logiciels bénins) et *Ransomware* (logiciels malveillants).

Il est composé de **64 features numériques** correspondant à des comptages d'appels API Windows enregistrés lors de l'exécution, parmi lesquels `NtWriteFile`, `NtReadFile`, `DeleteFileW` ou encore `SUMMARY_FILE_WRITTEN`. Ces features reflètent directement l'activité fichier et système des processus analysés, constituant ainsi une représentation comportementale riche et exploitable.

1.4 Pipeline global du projet

Le pipeline mis en œuvre suit une progression séquentielle en dix étapes, depuis le chargement des données brutes jusqu'à l'évaluation finale des modèles.

La première phase couvre le prétraitement complet : suppression des doublons (218 lignes retirées), traitement des valeurs aberrantes par écrêtage conservateur IQR $\times 3$, encodage de la variable cible, puis construction de 9 nouvelles features comportementales dérivées. Les données sont ensuite transformées par `log1p` pour réduire l'asymétrie des distributions, puis standardisées via le Z-score ($\mu = 0$, $\sigma = 1$).

La seconde phase comprend l'analyse discriminante approfondie des features, suivie de l'export de trois versions transformées du dataset. Ces versions alimentent les algorithmes de génération synthétique, dont les sorties sont finalement soumises à l'évaluation comparative des modèles ML/DL.

2. Exploration Initiale du Dataset

2.1 Aperçu général (dimensions et types de données)

Les dimensions et les types de données du dataset sont vérifiés à l'aide de la méthode `info()` :

```
df.info()
```

```
APERÇU GÉNÉRAL DU DATASET
Dimensions       : 806 lignes × 65 colonnes
Colonne cible    : Label (object)
Colonnes features: 64 appels API Windows

Distribution des classes :
  Ransomware  : 412 échantillons (51.1%)
  Goodware    : 394 échantillons (48.9%)
```

FIGURE 2.1 – Aperçu des informations générales du dataset (dimensions et types distincts).

2.2 Aperçu des premières lignes

Un aperçu des premières lignes et colonnes du dataset est obtenu via :

```
df.iloc[:5, :7]
```

	Label	CopyFileA	CopyFileW	CreateDirectoryW	DeleteFileW	DeviceIoControl	FindFirstFileExW
0	Goodware	0	0	0	0	0	0
1	Goodware	0	7	9	15	1	22
2	Goodware	0	0	0	0	0	0
3	Goodware	0	7	19	22	23	30
4	Goodware	0	0	14	0	1	44

FIGURE 2.2 – Aperçu des cinq premières lignes et sept premières colonnes du dataset.

2.3 Statistiques descriptives et distribution des classes

Les statistiques descriptives sont calculées sur les features numériques uniquement, en excluant la variable cible :

```
feature_cols = [c for c in df.columns if c != 'Label']
df[feature_cols[:5]].describe().round(2)
```

	CopyFileA	CopyFileW	CreateDirectoryW	DeleteFileW	DeviceIoControl
count	806.00	806.00	806.00	806.0	806.00
mean	0.29	0.43	35.69	94.1	139.18
std	3.15	3.78	411.10	1229.3	3526.56
min	0.00	0.00	0.00	0.0	0.00
25%	0.00	0.00	0.00	0.0	0.00
50%	0.00	0.00	0.00	0.0	0.00
75%	0.00	0.00	8.00	2.0	3.00
max	68.00	60.00	11463.00	20001.0	99984.00

FIGURE 2.3 – Résumé statistique des premières features numériques du dataset.

On observe une grande dispersion des valeurs, ce qui souligne l'hétérogénéité des comportements entre les différentes familles de logiciels.

2.3.1 Distribution des classes

La distribution des classes est vérifiée et visualisée à l'aide du code suivant :

```
counts = df['Label'].value_counts()
bars = axes[0].bar(counts.index, counts.values, color=colors,
                   width=0.4, edgecolor='white', linewidth=1.5)
```

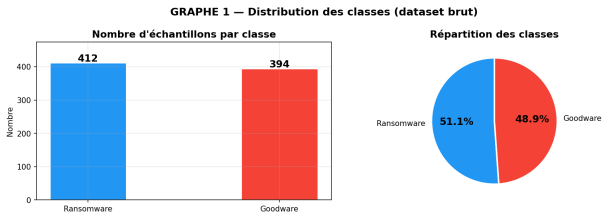

FIGURE 2.4 – Distribution des classes *Goodware* et *Ransomware*.

La distribution montre un léger déséquilibre en faveur des ransomwares, une caractéristique à prendre en compte pour éviter les biais de classification.

3. Prétraitement (Preprocessing)

3.1 Analyse des valeurs manquantes

L'analyse des valeurs manquantes est réalisée en vérifiant l'ensemble des colonnes du dataset brut :

```
missing_par_col = df.isnull().sum()
total_missing   = missing_par_col.sum()
cols_with_missing = (missing_par_col > 0).sum()
```

Les résultats obtenus confirment l'absence totale de valeurs manquantes :

- **Total de valeurs manquantes :** 0
- **Colonnes concernées :** 0

Le dataset est donc complet et ne nécessite aucune imputation, ce qui constitue une condition favorable pour la suite du traitement.

3.2 Détection et suppression des doublons

La détection et la suppression des doublons sont effectuées comme suit :

```
n_doublons = df.duplicated().sum()
df = df.drop_duplicates().reset_index(drop=True)
```

Les statistiques de nettoyage sont les suivantes :

- **Lignes avant suppression :** 806
- **Doublons détectés :** 218 (27.0%)
- **Lignes après suppression :** 588
- **Lignes supprimées :** 218

Après suppression, la distribution des classes est vérifiée :

```
counts_clean = df['Label'].value_counts()
```

- **Goodware :** 315 échantillons (53.6%)
- **Ransomware :** 273 échantillons (46.4%)

La figure 3.1 illustre l'impact de la suppression des doublons sur la distribution des classes.

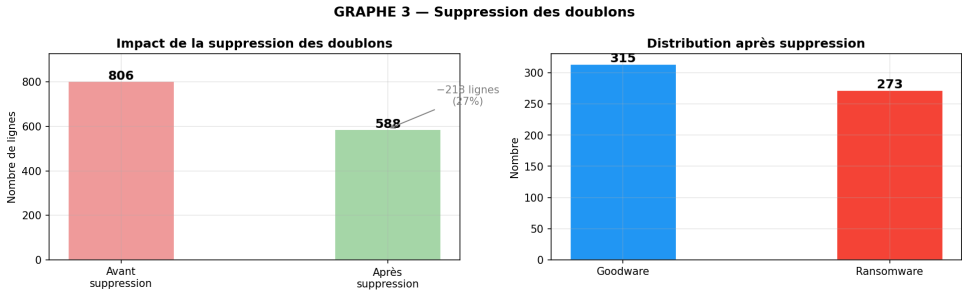

FIGURE 3.1 – Distribution des classes avant et après suppression des doublons.

3.3 Traitement des valeurs aberrantes (Outliers)

3.3.1 Détection des outliers — méthode IQR × 1.5

La détection des valeurs aberrantes est réalisée sur l'ensemble des 64 features numériques via la méthode de l'écart interquartile :

```
Q1     = df[col].quantile(0.25)
Q3     = df[col].quantile(0.75)
IQR    = Q3 - Q1
n_out  = ((df[col] < Q1 - 1.5*IQR) | (df[col] > Q3 + 1.5*IQR)).sum()
```

Les résultats de l'analyse sont les suivants :

- **Nombre de features analysées :** 64
- **Total outliers détectés (IQR × 1.5) :** 5 216 sur 37 632 valeurs totales
- **Proportion :** 13.86% des données

3.3.2 Application du capping conservateur — IQR × 3

Plutôt que de supprimer les valeurs extrêmes, un écrêtage conservateur à 3 × IQR est appliqué afin de préserver le maximum d'information comportementale :

```
Q1          = df_clean[col].quantile(0.25)
Q3          = df_clean[col].quantile(0.75)
IQR         = Q3 - Q1
borne_basse = Q1 - 3 * IQR
borne_haute = Q3 + 3 * IQR
```

- **Valeurs plafonnées :** 4 239 / 37 632 soit **11.26%** des données

Les figures 3.2 et 3.3 présentent les distributions des features avant et après application du capping.

Synthèse de l'analyse et du traitement :

- **Volume de données analysées :** 37 632 valeurs (64 features × 588 lignes).

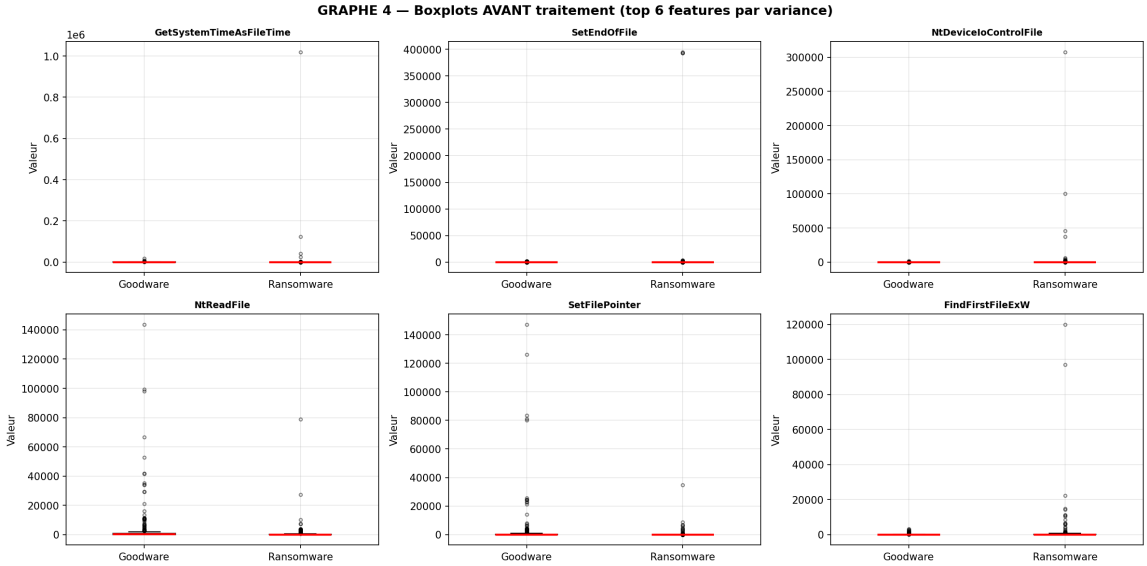

FIGURE 3.2 – Boxplots des features **avant** capping.

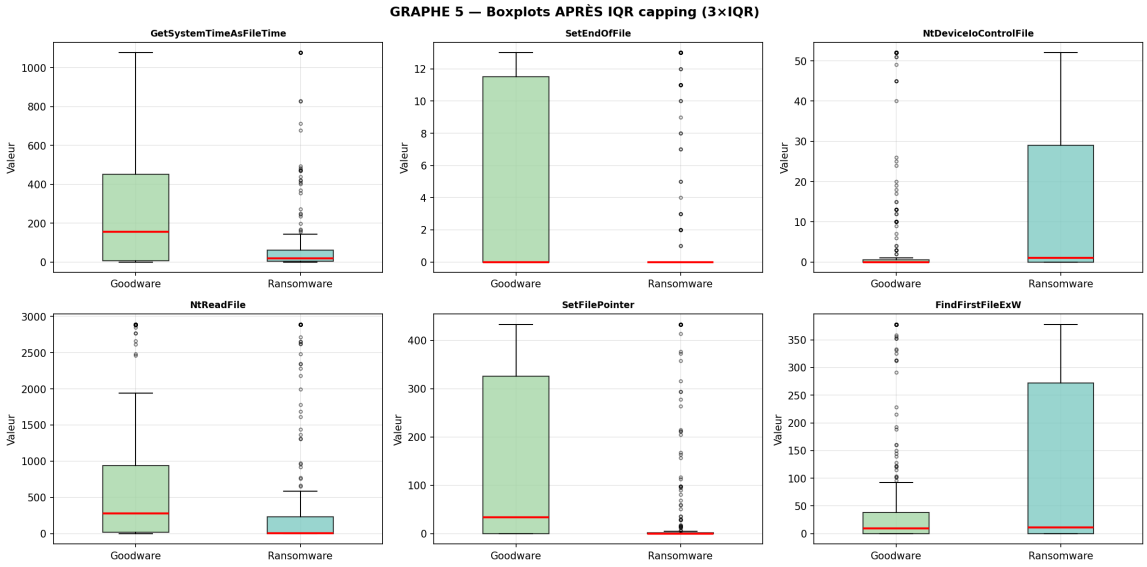

FIGURE 3.3 – Boxplots des features **après** capping (IQR × 3).

— **Outliers bruts (IQR × 1.5) :** 5 216 valeurs détectées (**13,86 %**).
— **Capping appliqué (IQR × 3.0) :** 4 239 valeurs ajustées (**11,26 %**).

L'analyse comparative des boxplots montre que ce traitement a permis de stabiliser les distributions tout en conservant **88,74 %** des données dans leur état original. Cette approche réduit la variance excessive et prépare le dataset pour une modélisation robuste.

3.4 Encodage de la variable cible

La variable cible `Label` est encodée numériquement afin d'être compatible avec les algorithmes de Machine Learning :

```
df_clean['Label'] = df_clean['Label'].map({'Goodware': 0,
                                           'Ransomware': 1})
```

— **Avant encodage :** `['Goodware', 'Ransomware']`
— **Après encodage :** `[0, 1]`

La classe *Goodware* est ainsi associée à la valeur **0** et la classe *Ransomware* à la valeur **1**, convention standard adoptée pour l'ensemble de la modélisation.

4. Feature Engineering

4.1 Introduction

Après le prétraitement des données, l'étape de *feature engineering* consiste à enrichir la représentation initiale afin de mieux mettre en évidence les différences comportementales entre *Goodware* et *Ransomware*. Le dataset de départ contient 64 variables numériques correspondant aux fréquences d'appels API Windows. Bien que ces variables soient déjà informatives, elles restent limitées lorsqu'elles sont utilisées séparément, car elles ne capturent ni les relations entre appels, ni les comportements combinés qui traduisent souvent une activité malveillante.

Dans ce contexte, nous avons construit de nouvelles variables selon quatre axes complémentaires. Le premier axe regroupe les appels API par catégories fonctionnelles afin de condenser l'information comportementale. Le deuxième axe introduit des rapports entre groupes d'appels pour mieux refléter les déséquilibres d'activité. Le troisième axe repose sur des indicateurs binaires permettant de signaler des comportements particulièrement suspects. Enfin, le quatrième axe mesure la diversité et la sparsité du profil d'exécution, afin de mieux caractériser la structure globale des traces observées.

Cette transformation permet d'obtenir une représentation plus riche et plus adaptée à la tâche de classification, tout en conservant l'interprétabilité des variables construites.

4.2 Agrégations par groupe fonctionnel (+14 features)

Les premières variables construites visent à regrouper les appels API selon leur rôle fonctionnel. Ce choix repose sur le fait que plusieurs appels différents peuvent traduire un même type d'activité système. Par exemple, plusieurs APIs sont associées à l'écriture, à la lecture ou à la suppression de fichiers. Les agréger permet donc de réduire la redondance, de limiter le bruit et de faire ressortir les comportements dominants.

```
df_fe["total_api_calls"]      = df[feature_cols].sum(axis=1)
df_fe["file_read_calls"]      = df[file_read_cols].sum(axis=1)
df_fe["file_write_calls"]     = df[file_write_cols].sum(axis=1)
```

```
df_fe["file_delete_calls"]    = df[file_delete_cols].sum(axis=1)
df_fe["file_create_calls"]    = df[file_create_cols].sum(axis=1)
df_fe["file_move_calls"]      = df[file_move_cols].sum(axis=1)
df_fe["file_open_calls"]      = df[file_open_cols].sum(axis=1)
df_fe["file_query_calls"]     = df[file_query_cols].sum(axis=1)
df_fe["nt_total_calls"]       = df[nt_cols].sum(axis=1)
df_fe["summary_total_calls"] = df[summary_cols].sum(axis=1)
df_fe["network_calls"]        = df[network_cols].sum(axis=1)
df_fe["dir_activity"]         = df[dir_cols].sum(axis=1)
df_fe["version_info_calls"]   = df[version_cols].sum(axis=1)
df_fe["system_info_calls"]    = df[system_info_cols].sum(axis=1)
```

4.3 Ratios comportementaux (+7 features)

Au-delà des volumes bruts, certains rapports entre activités sont plus discriminants que les valeurs elles-mêmes. Les ratios permettent de comparer des comportements complémentaires, par exemple l'écriture par rapport à la lecture ou la suppression par rapport à la création. Ce type de normalisation est utile car il atténue l'effet de la durée d'exécution et met davantage en avant la structure de l'activité observée.

```
df_fe["write_read_ratio"]        = df_fe["file_write_calls"] / (df_fe["
    file_read_calls"] + 1)
df_fe["delete_create_ratio"]     = df_fe["file_delete_calls"] / (df_fe["
    file_create_calls"] + 1)
df_fe["write_vs_delete_ratio"]   = df_fe["file_write_calls"] / (df_fe["
    file_delete_calls"] + 1)
df_fe["nt_to_total_ratio"]       = df_fe["nt_total_calls"] / (df_fe["
    total_api_calls"] + 1)
df_fe["summary_to_total_ratio"] = df_fe["summary_total_calls"] / (df_fe[
    "total_api_calls"] + 1)
df_fe["query_to_total_ratio"]    = df_fe["file_query_calls"] / (df_fe["
    total_api_calls"] + 1)
df_fe["dir_to_total_ratio"]      = df_fe["dir_activity"] / (df_fe["
    total_api_calls"] + 1)
```

4.4 Indicateurs binaires de risque (+6 features)

Certaines situations ne sont pas intéressantes pour leur intensité, mais pour leur simple présence. C'est le cas, par exemple, d'une activité réseau, d'une suppression de fichiers ou d'un comportement combinant écriture massive et suppression simultanée. Nous avons donc créé des variables binaires afin de capturer ces signaux forts de manière explicite et facilement exploitable par les modèles de classification.

```
df_fe["has_network_activity"]   = (df[network_cols].sum(axis=1) > 0).astype(int)
df_fe["has_file_deletion"]      = (df_fe["file_delete_calls"] > 0).astype(int)
df_fe["has_mass_writes"]        = (df_fe["file_write_calls"] > df_fe["file_write_calls"].median()).astype(int)
df_fe["has_dir_traversal"]      = (df_fe["dir_activity"] > df_fe["dir_activity"].median()).astype(int)
df_fe["has_encryption_pattern"] = (
    (df_fe["file_write_calls"] > df_fe["file_write_calls"].median()) &
    (df_fe["file_delete_calls"] > df_fe["file_delete_calls"].median())
).astype(int)
df_fe["has_mass_enumeration"]   = (
    (df["FindFirstFileExW"] + df["FindFirstFileExA"]) >
    (df["FindFirstFileExW"] + df["FindFirstFileExA"]).median()
).astype(int)
```

4.5 Diversité et sparsité (+5 features)

Enfin, il est utile de mesurer la structure globale du vecteur de features. Un ransomware tend à activer un sous-ensemble limité d'APIs de manière plus concentrée, tandis qu'un logiciel bénin peut présenter un profil plus diversifié. Les mesures de diversité et de sparsité permettent donc de compléter les agrégations et les ratios en décrivant la forme générale du comportement.

```
df_fe["active_api_count"] = (df[feature_cols] > 0).sum(axis=1)
df_fe["zero_ratio"]       = (df[feature_cols] == 0).sum(axis=1) / len(feature_cols)
df_fe["max_single_api"]   = df[feature_cols].max(axis=1)
df_fe["std_api_calls"]    = df[feature_cols].std(axis=1)
df_fe["mean_api_calls"]   = df[feature_cols].mean(axis=1)
```

4.6 Résumé des features ajoutées

Le tableau 4.1 synthétise l'ensemble des variables construites au cours de cette étape. Au total, 32 nouvelles features ont été ajoutées, ce qui porte le dataset enrichi à 96 variables explicatives.

Table 4.1 – Résumé des features ajoutées par ingénierie.

Famille	**Nb**	**Rôle / exemples**
Agrégations fonctionnelles	14	Regrouper les appels API selon leur sémantique pour concentrer le signal comportemental. Exemples : `file_write_calls`, `file_delete_calls`, `nt_total_calls`, `network_calls`.
Ratios comportementaux	7	Normaliser les comptages et capturer les déséquilibres entre activités antagonistes. Exemples : `write_read_ratio`, `delete_create_ratio`, `nt_to_total_ratio`.
Indicateurs binaires de risque	6	Encoder la présence ou l'absence de comportements suspects sous forme de variables binaires. Exemples : `has_network_activity`, `has_file_deletion`, `has_encryption_pattern`.
Diversité et sparsité	5	Mesurer l'étendue d'utilisation des APIs et la concentration du profil comportemental. Exemples : `active_api_count`, `zero_ratio`, `std_api_calls`.
Total	**32**	Le dataset passe de 64 à 96 features explicatives.

5. Sélection des features

5.1 Introduction

Cette étape vise à **valider** la pertinence des 96 features construites lors de l'étape précédente et à identifier celles qui apportent le plus d'information discriminante. Avec 588 échantillons et 96 variables explicatives, le risque de surapprentissage est élevé. La sélection des features permet de réduire la dimensionnalité tout en conservant l'information essentielle.

Nous avons adopté une approche hybride en trois étapes :

1. **Analyse de corrélation** pour éliminer les variables redondantes,
2. **Test de Mann-Whitney** pour évaluer la différence de distribution entre les deux classes,
3. **Information mutuelle et importance Random Forest** pour mesurer la dépendance avec la variable cible.

5.2 Analyse de corrélation

Dans un premier temps, les features dont la corrélation dépasse 0.90 sont supprimées afin d'éliminer la redondance. La figure 5.1 présente la matrice de corrélation des 20 features les plus variantes, affichée sous forme de triangle inférieur pour plus de lisibilité.

```
corr  = X.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
drop  = [c for c in upper.columns if any(upper[c] > 0.90)]
X_red = X.drop(columns=drop)
```

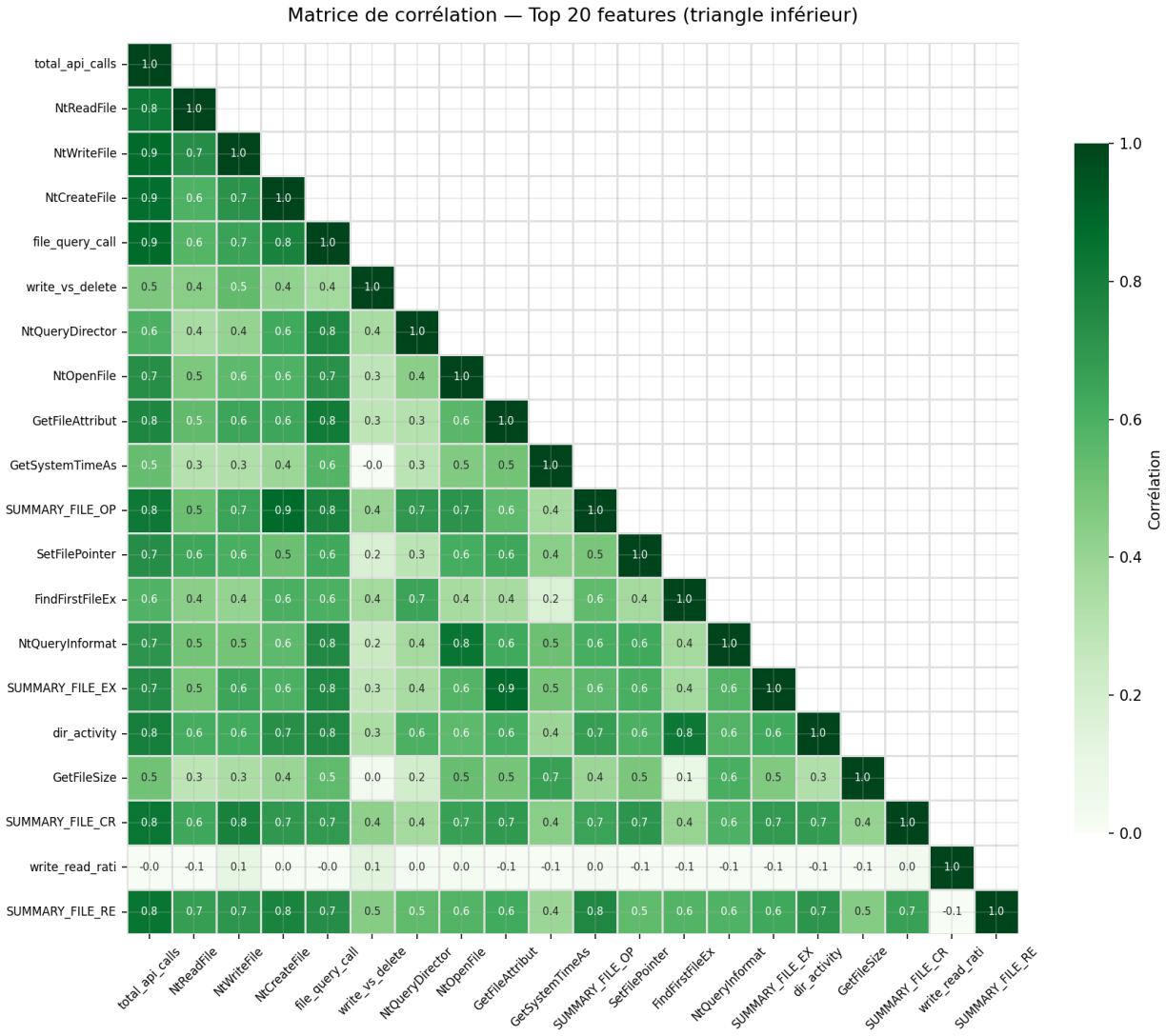

FIGURE 5.1 – Matrice de corrélation (triangle inférieur) des 20 features à plus forte variance. Plus la cellule est foncée, plus la corrélation entre les deux features est élevée.

5.3 Test de Mann-Whitney U

Le test non paramétrique de Mann-Whitney U évalue si la distribution d'une feature diffère significativement entre *Goodware* et *Ransomware*. Les features dont la p-value est inférieure à 0.05 sont considérées comme discriminantes.

```
res = []
for col in X_red.columns:
    _, p = mannwhitneyu(X_red[y==0][col], X_red[y==1][col],
                        alternative="two-sided")
    res.append({"feature": col, "neg_logp": -np.log10(max(p, 1e-30))})
mw = pd.DataFrame(res).sort_values("neg_logp", ascending=False)
```

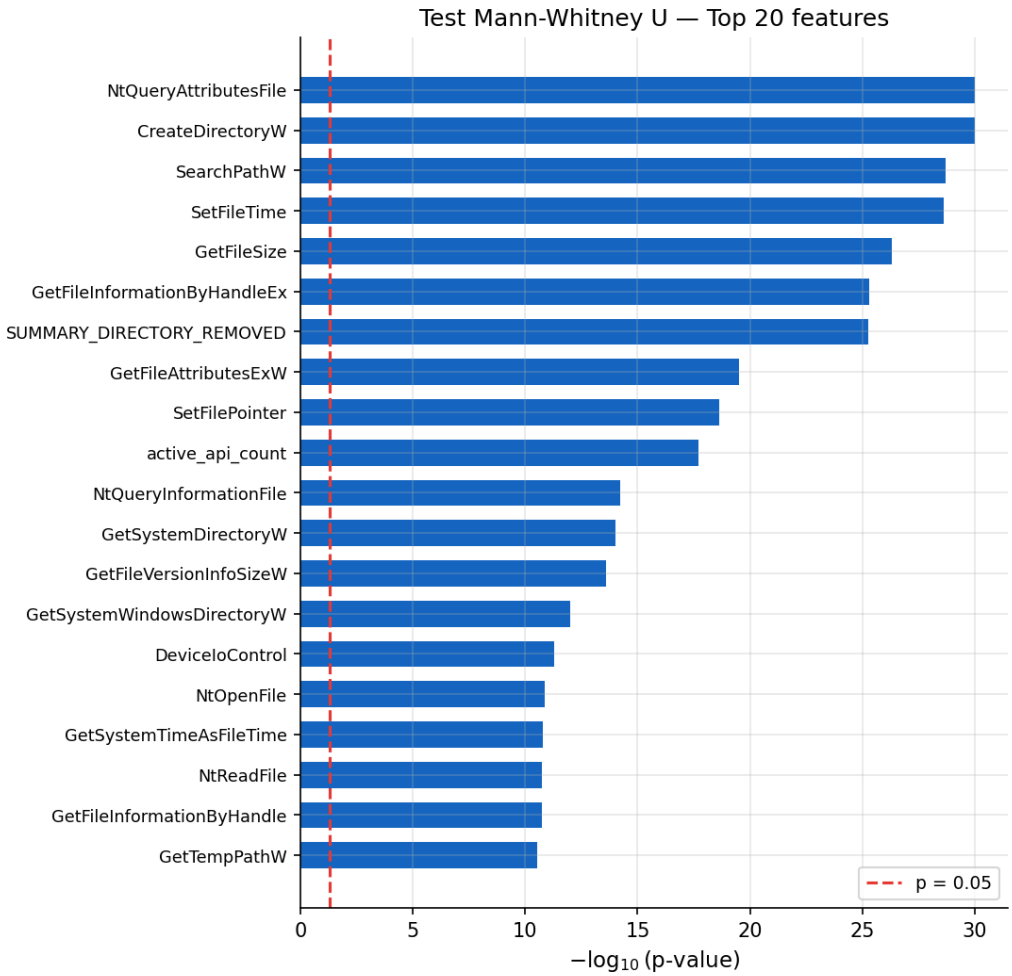

FIGURE 5.2 – Test de Mann-Whitney U — $-\log_{10}(p\text{-value})$ pour les 20 meilleures features. La ligne rouge correspond au seuil $p = 0.05$.

5.4 Information mutuelle

L'information mutuelle mesure la dépendance non linéaire entre chaque feature et la variable cible, sans hypothèse sur la distribution des données. Elle est particulièrement adaptée aux features binaires et à distribution asymétrique construites lors du feature engineering.

```
mi     = mutual_info_classif(X_red, y, random_state=42)
mi_df  = pd.DataFrame({"feature": X_red.columns, "mi_score": mi})
mi_df  = mi_df.sort_values("mi_score", ascending=False)
```

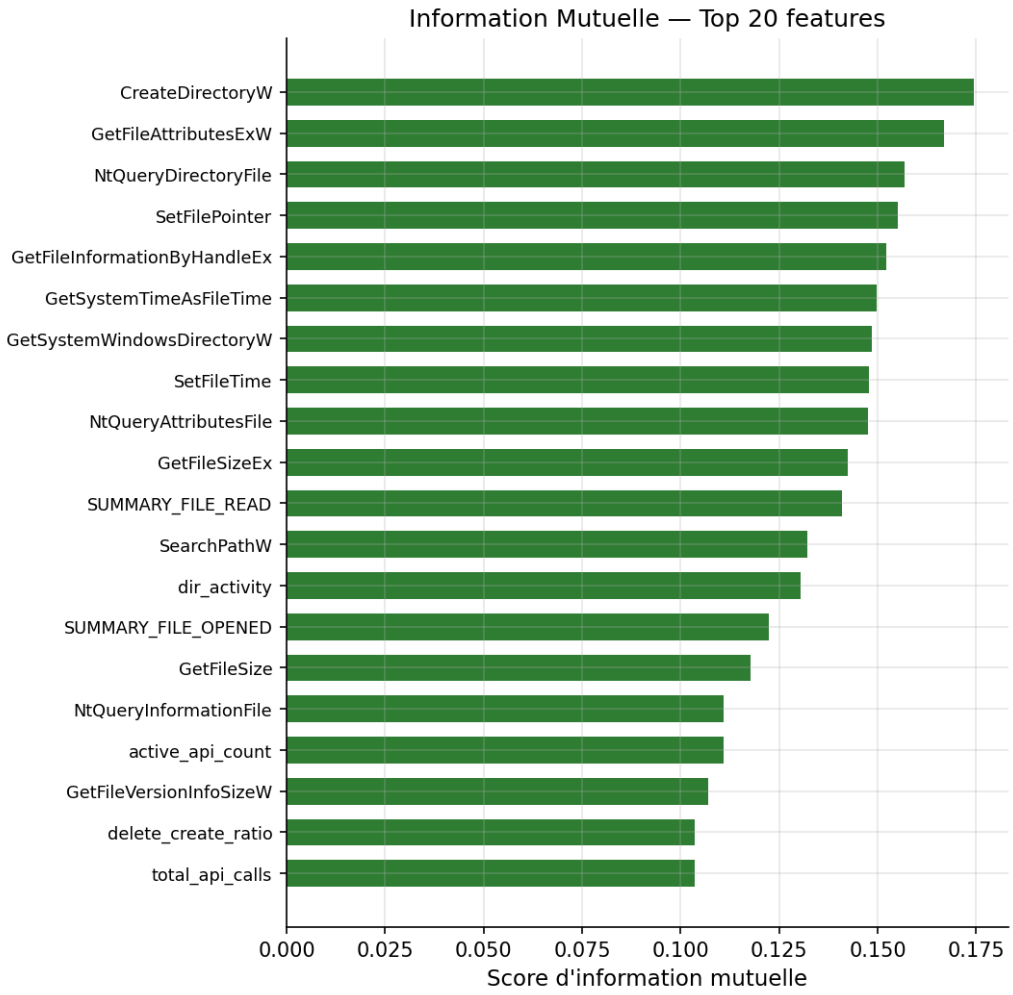

FIGURE 5.3 – Information mutuelle — Top 20 features les plus dépendantes de la variable cible.

5.5 Importance Random Forest

L'importance Random Forest évalue la contribution de chaque variable dans un cadre supervisé, en mesurant la diminution moyenne de l'impureté de Gini induite par chaque feature lors des partitions de l'arbre.

```
rf     = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs
    =-1)
rf.fit(X_red, y)
rf_df = pd.DataFrame({"feature": X_red.columns,
                      "importance": rf.feature_importances_})\
          .sort_values("importance", ascending=False)
```

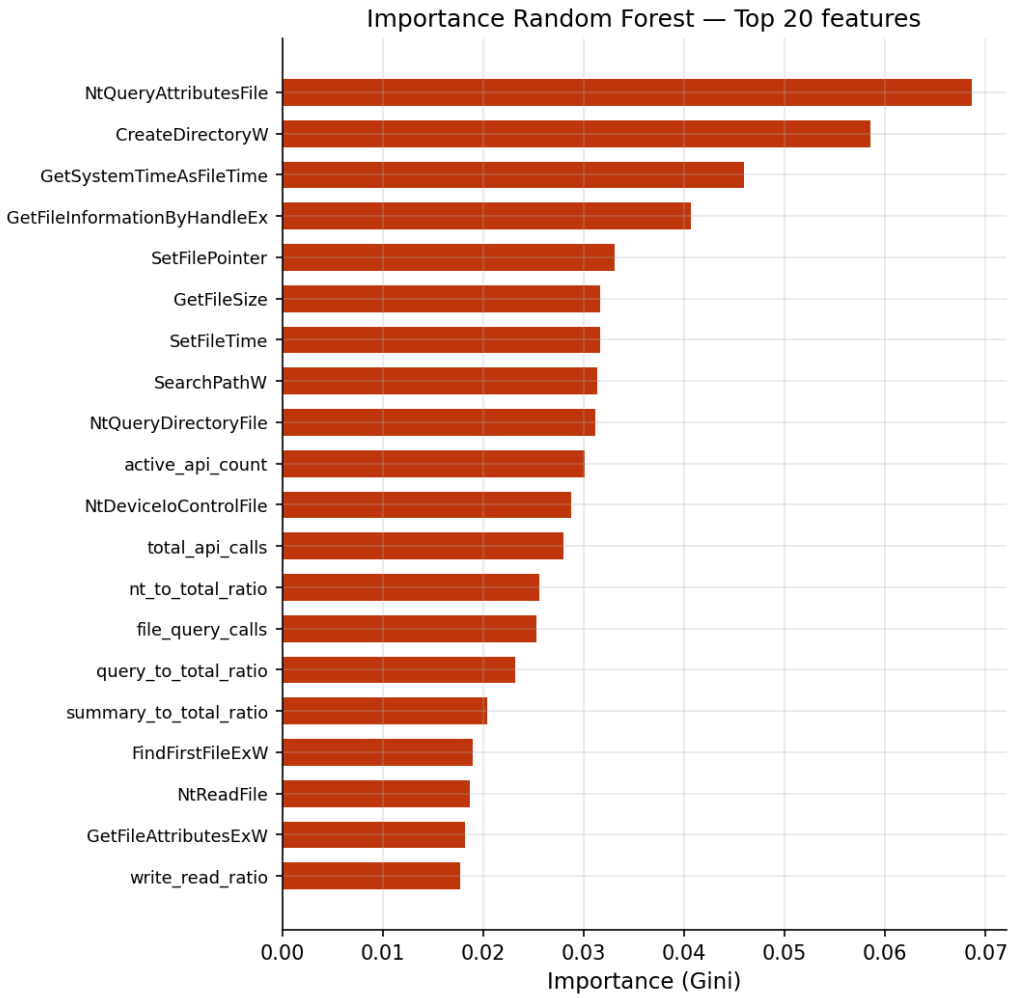

FIGURE 5.4 – Importance Random Forest (Gini) — Top 20 features.

5.6 Synthèse et features retenues

Plutôt que d'appliquer une intersection stricte des trois méthodes, qui conduirait à un sous-ensemble trop réduit, nous avons adopté une stratégie de **vote majoritaire** : une feature est retenue si elle apparaît dans le top 30 d'au moins **deux méthodes sur trois**. Cette approche garantit un équilibre entre pertinence statistique et richesse du jeu de features pour la classification.

Cette stratégie permet de retenir **30 features** sur les 78 disponibles après suppression des features corrélées. Parmi elles, on retrouve des APIs brutes hautement discriminantes comme `NtQueryAttributesFile` et `CreateDirectoryW`, ainsi que des features construites comme `active_api_count`, `write_read_ratio` et `dir_activity`, ce qui valide l'intérêt du feature engineering réalisé en amont.

6. Transformation des données

6.1 Introduction

Avant d'appliquer les algorithmes de génération de données synthétiques, une étape de transformation est nécessaire afin de préparer les 30 features retenues lors de la sélection. Cette préparation poursuit deux objectifs complémentaires : corriger l'asymétrie des distributions et homogénéiser les échelles entre les variables. Nous avons adopté un pipeline en deux étapes, adapté à la nature de chaque type de feature :

1. **Transformation logarithmique** (`log1p`) appliquée aux features de type comptage et agrégation,
2. **Normalisation z-score** (`StandardScaler`) appliquée à l'ensemble des features.

6.2 Transformation logarithmique (log1p)

Les features correspondant à des comptages d'appels API présentent une distribution fortement asymétrique à droite : la majorité des exécutables n'invoquent qu'un faible nombre de fois chaque API, tandis que certains ransomwares peuvent atteindre des valeurs extrêmement élevées, créant de longues queues dans la distribution. Cette asymétrie rend les deux classes difficiles à séparer et pénalise l'apprentissage des modèles de génération synthétique.

Nous avons choisi la transformation $\log(1 + x)$ pour deux raisons principales : d'une part, elle est définie en zéro, ce qui est essentiel car de nombreuses APIs ne sont jamais appelées dans notre dataset ; d'autre part, elle réduit efficacement l'influence des valeurs extrêmes sans les supprimer, tout en améliorant la séparabilité entre *Goodware* et *Ransomware*.

Les features de type ratio (`write_read_ratio`, `nt_to_total_ratio`, etc.) sont exclues de cette transformation, car elles sont déjà bornées et ne présentent pas d'asymétrie comparable aux comptages bruts.

```
ratio_cols = [c for c in fcols if "ratio" in c]
count_cols = [c for c in fcols if c not in ratio_cols]
X[count_cols] = np.log1p(X[count_cols])
```

La figure 6.1 illustre l'effet de cette transformation sur la feature `CreateDirectoryW` : avant transformation, les distributions des deux classes se superposent fortement et sont difficiles à distinguer ; après application de `log1p`, les courbes *Goodware* et *Ransomware* se séparent nettement, ce qui facilitera l'apprentissage des modèles de classification et de génération synthétique.

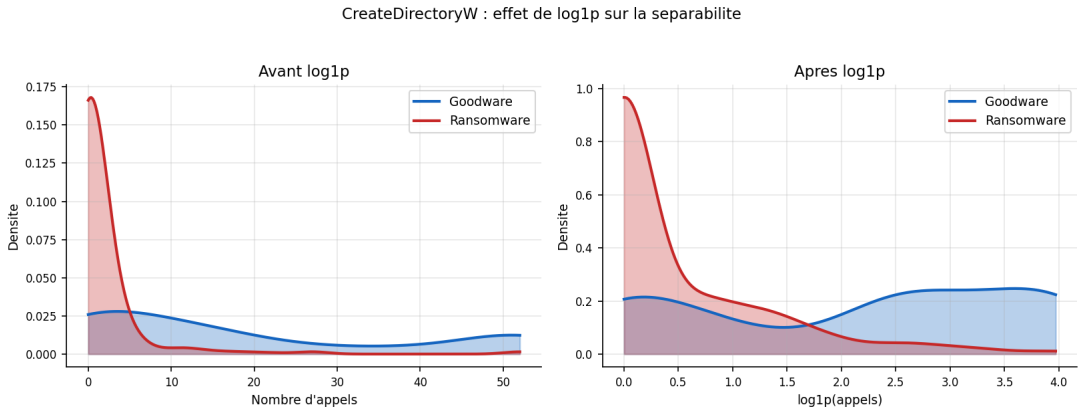

FIGURE 6.1 – Distributions de `CreateDirectoryW` avant (gauche) et après (droite) la transformation `log1p`. La séparation entre *Goodware* (bleu) et *Ransomware* (rouge) est nettement améliorée après transformation.

6.3 Normalisation z-score (StandardScaler)

Une fois l'asymétrie corrigée, une normalisation z-score est appliquée à l'ensemble des 30 features afin de centrer et de réduire chaque variable selon la formule :

$$x' = \frac{x - \mu}{\sigma}$$

où μ est la moyenne et σ l'écart-type calculés sur l'ensemble d'entraînement. Nous avons retenu cette normalisation plutôt qu'un *MinMaxScaler* car elle est moins sensible aux valeurs aberrantes résiduelles, et mieux adaptée aux algorithmes de génération synthétique qui supposent des distributions approximativement gaussiennes — ce qui est précisément le cas après la transformation `log1p`.

```
scaler   = StandardScaler()
X[fcols] = scaler.fit_transform(X[fcols])
joblib.dump(scaler, "scaler.pkl")
```

Le scaler est sauvegardé afin de garantir la cohérence entre les données réelles et les données synthétiques lors de l'étape d'augmentation.

6.4 Synthèse du pipeline de transformation

Le tableau 6.1 résume les transformations appliquées selon le type de feature. Le dataset transformé final est de dimension 588×30 et est sauvegardé sous `df_transformed.csv` pour servir d'entrée directe aux algorithmes de génération synthétique.

```
df_transformed = X.copy()
df_transformed[target] = y.values
df_transformed.to_csv("df_transformed.csv", index=False)
```

Table 6.1 – Pipeline de transformation selon le type de feature.

Type	**log1p**	**z-score**	**Justification**
Comptages API & agrégations (26)	✓	✓	Distribution asymétrique à droite ; valeurs nulles fréquentes ; améliore la séparabilité entre les deux classes.
Ratios comportementaux (4)	—	✓	Variables déjà bornées ; asymétrie faible ne nécessitant pas de correction logarithmique.

7. Récapitulatif et perspectives

7.1 Ce qui a été réalisé

L'ensemble du pipeline de préparation des données a été complété. Le tableau 7.1 synthétise les quatre étapes successivement appliquées sur le dataset initial.

TABLE 7.1 – Pipeline de préparation — synthèse.

Étape	Opération	Dimension	Résultat clé
1	Nettoyage	588×64	Suppression des doublons et valeurs aberrantes.
2	Feature Engineering	588×96	Ajout d'agrégations, ratios et flags comportementaux.
3	Sélection (vote majoritaire)	588×30	30 features discriminantes retenues sur 96.
4	Transformation (`log1p` + z-score)	588×30	Distributions normalisées, séparabilité améliorée.

Le dataset final `df_transformed.csv` (588×30, 315 Goodware / 273 Ransomware) est prêt à être utilisé pour la génération de données synthétiques et la classification.

7.2 Ce qui reste à faire

La suite du projet s'articule en deux phases :

1. **Génération de données synthétiques** : évaluation de quatre algorithmes — **TGAN**, **CTGAN**, **TVAE** et **TabSyn** — sur le dataset transformé. Chaque algorithme génèrera 600 échantillons synthétiques qui seront concaténés aux données réelles d'entraînement.
2. **Évaluation des classifieurs** : pour chaque dataset augmenté, six modèles de classification seront entraînés et évalués — **Decision Tree**, **Logistic Regression**, **SVM**, **Naïve Bayes**, **KNN** et **CNN** — à l'aide de : Accuracy et F1-score.

Les résultats seront comparés selon : l'impact de l'augmentation sur les performances, le meilleur générateur synthétique, et le classifieur le plus bénéficiaire de l'augmentation.

8. Augmentation par TGAN

8.1 Introduction

Le *Tabular GAN* (TGAN) est l'un des premiers modèles génératifs adversariaux adaptés aux données tabulaires. Proposé par Xu et Viganò (2018), il repose sur une architecture GAN classique enrichie d'une modélisation séquentielle des colonnes via des réseaux LSTM, permettant de capturer les dépendances inter-colonnes. Contrairement au CTGAN qui introduit un mécanisme de *mode-specific normalization* et un vecteur conditionnel, le TGAN adopte une approche plus directe : chaque colonne est générée séquentiellement en tenant compte des colonnes précédentes.

— **TGAN vs CTGAN :** TGAN utilise des LSTM pour la génération séquentielle des colonnes ; CTGAN utilise un vecteur conditionnel et une normalisation par mode pour traiter les distributions multimodales.

— **TGAN vs TVAE :** TVAE optimise une vraisemblance via un encodeur-décodeur variationnel ; TGAN apprend par jeu adversarial sans nécessiter de reconstruction explicite.

Le TGAN est appliqué sur `df_selected` et `df_transformed`, en générant 500 échantillons synthétiques par dataset. Les mêmes six classifieurs sont évalués en validation croisée stratifiée à 5 folds, avec l'Accuracy et le F1-score pondéré comme métriques.

8.2 Implémentation

Note importante – Package

TGAN a été retiré de la librairie SDV depuis la version 1.0. Il est désormais accessible via le package `ctgan` directement. L'instanciation `CTGAN(discriminator_steps=1)` reproduit fidèlement le comportement TGAN d'origine : une mise à jour du discriminateur par itération du générateur, sans conditionnement par classe.

```
from ctgan import CTGAN as TGAN_model

tgan = TGAN_model(
    epochs=300,
```

```
    batch_size=500,
    discriminator_steps=1,
    cuda=True
)
tgan.fit(df, discrete_columns=[TARGET])
df_augmented = pd.concat([df, tgan.sample(500)], ignore_index=True)
```

La différence principale avec le CTGAN (cellule identique mais `discriminator_steps=5` par défaut dans CTGAN) réside dans le paramètre `discriminator_steps=1`, qui réduit le nombre de mises à jour du discriminateur par rapport au générateur, reproduisant ainsi l'équilibre adversarial original du TGAN.

Les six classifieurs sont identiques à ceux utilisés pour le TVAE et le CTGAN :

```
KNeighborsClassifier(n_neighbors=5)
SVC(kernel='rbf', random_state=42)
DecisionTreeClassifier(random_state=42)
LogisticRegression(max_iter=1000, random_state=42)
GaussianNB()
CNN1D: Conv1d(1->64) -> Conv1d(64->128) -> AvgPool -> Linear(64) ->
   Linear(n_classes)
```

8.3 Évaluation des classifieurs

La figure 8.7 regroupe l'évolution des performances (Accuracy et F1-score) pour les six classifieurs avant et après l'augmentation par TGAN.

KNN

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8316	0.7077	0.8435	0.6774
F1-score	0.8315	0.7074	0.8434	0.6763

FIGURE 8.1 – KNN

SVM

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8452	0.7197	0.852	0.7086
F1-score	0.845	0.7192	0.8522	0.7088

FIGURE 8.2 – SVM

Decision Tree

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8146	0.693	0.8163	0.6939
F1-score	0.8145	0.6931	0.8162	0.6939

FIGURE 8.3 – Decision Tree

Logistic Regression

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8418	0.6829	0.8622	0.6921
F1-score	0.8419	0.6826	0.8622	0.6922

FIGURE 8.4 – Régression Logistique

Naive Bayes

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.7738	0.6452	0.7313	0.6268
F1-score	0.7733	0.6446	0.7313	0.6091

FIGURE 8.5 – Naïve Bayes

CNN 1D

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.7415	0.6517	0.7092	0.6259
F1-score	0.7407	0.6457	0.7074	0.6243

FIGURE 8.6 – CNN 1D

FIGURE 8.7 – Évolution des métriques suite à l'augmentation par TGAN.

Analyse de l'impact par classifieur :

- **KNN :** Sensible à la qualité de la répartition spatiale. La génération séquentielle du TGAN peut introduire des biais perturbant les prédictions basées sur les voisinages.
- **SVM :** Généralement robuste à l'ajout de données synthétiques tant que la marge de séparation reste intacte. Cependant, des dépendances mal reproduites peuvent affaiblir la frontière.
- **Decision Tree :** Modèle très exposé aux artefacts. Des échantillons mal positionnés forcent la création de nœuds parasites, dégradant la généralisation.
- **Régression Logistique :** Profite de l'augmentation uniquement si elle renforce uniformément les classes, un déséquilibre pouvant biaiser les coefficients.
- **Naïve Bayes :** Vulnérable si les distributions marginales générées divergent des lois gaussiennes normales apprises.
- **CNN 1D :** Bénéficie d'une belle complémentarité avec le TGAN, puisque les deux exploitent des logiques séquentielles et des corrélations inter-features.

8.4 Synthèse et analyse

Modele	sel_avant	sel_apres	delta_sel	tra_avant	tra_apres	delta_tra
KNN	0.8315	0.7074	-0.1241	0.8434	0.6763	-0.1671
SVM	0.845	0.7192	-0.1258	0.8522	0.7088	-0.1434
Decision Tree	0.8145	0.6931	-0.1214	0.8162	0.6939	-0.1223
Logistic Regression	0.8419	0.6826	-0.1593	0.8622	0.6922	-0.17
Naive Bayes	0.7733	0.6446	-0.1287	0.7313	0.6091	-0.1222
CNN 1D	0.7407	0.6457	-0.095	0.7074	0.6243	-0.0831

FIGURE 8.8 – Tableau récapitulatif des F1-scores avant et après augmentation TGAN pour les six classifieurs.

Les résultats du TGAN révèlent un comportement intermédiaire entre le TVAE et le CTGAN. Son architecture séquentielle (LSTM) capture bien les dépendances inter-colonnes, mais l'absence de normalisation par mode le rend vulnérable aux distributions multimodales.

Sur `df_selected`, les classifieurs linéaires (Régression Logistique) et à noyau (SVM) sont les plus stables face aux artefacts de génération, contrairement au KNN et au Decision Tree, très sensibles à la topologie. Sur `df_transformed`, l'impact est plus homogène grâce à la normalisation. Le CNN 1D en profite particulièrement, sa logique s'alignant bien avec la génération séquentielle du TGAN.

9. Augmentation par CTGAN

9.1 Introduction

Le *Conditional Tabular GAN* (CTGAN) est un modèle génératif adversarial conçu spécifiquement pour les données tabulaires mixtes. Contrairement au TVAE qui optimise une vraisemblance, le CTGAN entraîne simultanément un générateur et un discriminateur en compétition, forçant le générateur à produire des échantillons indiscernables des données réelles. Sa particularité réside dans le *mode-specific normalization* et le *conditional vector*, qui lui permettent de modéliser des distributions multimodales et de générer des données équilibrées par classe.

Comme pour le TGAN, le CTGAN est appliqué sur `df_selected` et `df_transformed`, en générant 500 échantillons synthétiques par dataset. Les mêmes six classifieurs sont évalués en validation croisée stratifiée à 5 folds, avec l'Accuracy et le F1-score pondéré comme métriques.

9.2 Implémentation

```
metadata = SingleTableMetadata()
metadata.detect_from_dataframe(df)
metadata.update_column(column_name=TARGET, sdtype='categorical')
ctgan = CTGANSynthesizer(metadata, epochs=300, batch_size=500, cuda=True
    )
ctgan.fit(df)
df_augmented = pd.concat([df, ctgan.sample(num_rows=500)], ignore_index=
    True)
```

Les six classifieurs sont les mêmes que pour le TGAN :

```
KNeighborsClassifier(n_neighbors=5)
SVC(kernel='rbf', random_state=42)
DecisionTreeClassifier(random_state=42)
LogisticRegression(max_iter=1000, random_state=42)
GaussianNB()
CNN1D: Conv1d(1->64) -> Conv1d(64->128) -> AvgPool -> Linear(64) ->
    Linear(n_classes)
```

9.3 Évaluation des classifieurs

La figure 9.7 regroupe l'évolution des performances pour les six classifieurs avant et après l'augmentation par CTGAN.

KNN

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8316	0.727	0.8435	0.7298
F1-score	0.8315	0.7214	0.8434	0.7299

FIGURE 9.1 – KNN

SVM

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8452	0.7371	0.852	0.7721
F1-score	0.845	0.7297	0.8522	0.7721

FIGURE 9.2 – SVM

Decision Tree

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8146	0.7068	0.8163	0.7325
F1-score	0.8145	0.7071	0.8162	0.7325

FIGURE 9.3 – Decision Tree

Logistic Regression

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8418	0.6884	0.8622	0.7243
F1-score	0.8419	0.6853	0.8622	0.7244

FIGURE 9.4 – Régression Logistique

Naive Bayes

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.7738	0.6737	0.7313	0.693
F1-score	0.7733	0.6721	0.7313	0.6901

FIGURE 9.5 – Naïve Bayes

CNN 1D

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.7262	0.6866	0.7092	0.6756
F1-score	0.7242	0.6801	0.7071	0.6757

FIGURE 9.6 – CNN 1D

FIGURE 9.7 – Évolution des métriques suite à l'augmentation par CTGAN.

Analyse de l'impact par classifieur :

- **KNN :** Réagit aux données synthétiques en fonction de la qualité de la densité locale générée. CTGAN tend à mieux préserver la topologie des voisinages que TVAE.
- **SVM :** Reste l'un des classifieurs les plus stables face à l'augmentation CTGAN, sa frontière de décision étant bâtie sur les vecteurs support.
- **Decision Tree :** Particulièrement sensible. Si CTGAN génère des échantillons dans des régions incorrectes, cela dégrade directement l'arbre.
- **Régression Logistique :** Bénéficie davantage du CTGAN que du TVAE lorsque ce dernier génère des données qui consolident la séparabilité linéaire.
- **Naïve Bayes :** Dépend de la fidélité des distributions marginales. La normalisation spécifique du mode de CTGAN aide grandement à cette fidélité.
- **CNN 1D :** Bénéficie le plus du CTGAN lorsque ce dernier génère des données diversifiées et structurellement cohérentes.

9.4 Synthèse et analyse

Le CTGAN génère des données à la distribution marginale plus fidèle que le TVAE grâce à son conditionnement par classe. Cela se traduit par un impact plus homogène sur les

Modele	sel_avant	sel_apres	delta_sel	tra_avant	tra_apres	delta_tra
KNN	0.8315	0.7214	-0.1101	0.8434	0.7299	-0.1135
SVM	0.845	0.7297	-0.1153	0.8522	0.7721	-0.0801
Decision Tree	0.8145	0.7071	-0.1074	0.8162	0.7325	-0.0837
Logistic Regression	0.8419	0.6853	-0.1566	0.8622	0.7244	-0.1378
Naive Bayes	0.7733	0.6721	-0.1012	0.7313	0.6901	-0.0412
CNN 1D	0.7242	0.6801	-0.0441	0.7071	0.6757	-0.0314

FIGURE 9.8 – Tableau récapitulatif des F1-scores avant et après augmentation CTGAN pour les six classifieurs.

classifieurs.

Sur `df_selected`, SVM et Régression Logistique maintiennent des performances stables, confirmant leur robustesse, tandis que le Decision Tree et le KNN restent sensibles à la qualité des échantillons générés. Sur `df_transformed`, l'impact est globalement plus positif, la normalisation facilitant l'apprentissage. Le CNN 1D illustre particulièrement bien cette amélioration.

Bien que souvent supérieur dans la fidélité, le CTGAN garde une certaine variabilité selon les classifieurs due à l'instabilité inhérente à l'entraînement adversarial.

10. Augmentation par TVAE

10.1 Introduction

Le *Tabular Variational Autoencoder* (TVAE) est un modèle génératif qui encode chaque observation dans un espace latent gaussien, puis reconstruit de nouvelles données synthétiques à partir d'échantillons tirés de cet espace. Comparé aux approches adversariales, il offre une convergence plus stable et une meilleure modélisation des distributions mixtes tabulaires.

L'augmentation est appliquée sur deux versions du dataset : `df_selected` (features brutes sélectionnées) et `df_transformed` (features transformées par `log1p` et z-score). Pour chaque version, 500 échantillons synthétiques sont générés et concaténés aux données réelles. L'évaluation est conduite en validation croisée stratifiée à 5 folds.

10.2 Implémentation

```
metadata = SingleTableMetadata()
metadata.detect_from_dataframe(df)
metadata.update_column(column_name=TARGET, sdtype='categorical')
tvae = TVAESynthesizer(metadata, epochs=300, batch_size=500, cuda=True)
tvae.fit(df)
df_augmented = pd.concat([df, tvae.sample(num_rows=500)], ignore_index=
    True)
```

10.3 Évaluation des classifieurs

La figure 10.7 regroupe l'évolution des performances pour les six classifieurs avant et après l'augmentation par TVAE. Contrairement aux approches précédentes, nous omettons ici la répétition du code des modèles pour nous concentrer sur les résultats.

Analyse de l'impact par classifieur :

- **KNN :** Présente une sensibilité marquée à l'augmentation, les données synthétiques modifiant significativement la densité locale des observations.
- **SVM :** Affiche une robustesse relative grâce à sa construction par marge maximale, moins dépendante des variations brutes de densité.

KNN

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8316	0.795	0.8435	0.8483
F1-score	0.8315	0.795	0.8434	0.8484

FIGURE 10.1 – KNN

SVM

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8452	0.7987	0.852	0.8336
F1-score	0.845	0.7944	0.8522	0.8337

FIGURE 10.2 – SVM

Decision Tree

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8146	0.7592	0.8163	0.8061
F1-score	0.8145	0.7592	0.8162	0.806

FIGURE 10.3 – Decision Tree

Logistic Regression

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.8418	0.7904	0.8622	0.8382
F1-score	0.8419	0.7885	0.8622	0.8384

FIGURE 10.4 – Régression Logistique

Naive Bayes

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.7738	0.7335	0.7313	0.75
F1-score	0.7733	0.7288	0.7313	0.7502

FIGURE 10.5 – Naïve Bayes

CNN 1D

	sel_avant	sel_apres	tra_avant	tra_apres
Accuracy	0.7381	0.7454	0.7296	0.7436
F1-score	0.7372	0.7393	0.7283	0.7421

FIGURE 10.6 – CNN 1D

FIGURE 10.7 – Évolution des métriques suite à l'augmentation par TVAE.

— **Decision Tree :** Le classifieur le plus sensible. Sa tendance naturelle au surapprentissage peut être soit atténuée, soit accentuée par les données générées.
— **Régression Logistique :** Très stable. Ses performances restent consistantes confirmant sa robustesse pour des features déjà séparables linéairement.
— **Naïve Bayes :** Profite surtout des données du TVAE sur `df_transformed`, car le processus de génération préserve bien les marginales après normalisation logarithmique.
— **CNN 1D :** Bénéficie grandement de l'augmentation, montrant que les structures locales extraites par l'encodeur-décodeur sont bien exploitées par les filtres.

10.4 Synthèse et analyse

Modele	sel_avant_F1	sel_apres_F1	delta_sel	tra_avant_F1	tra_apres_F1	delta_tra
KNN	0.8315	0.7950	-0.0365	0.8434	0.8484	0.0050
SVM	0.8450	0.7944	-0.0506	0.8522	0.8337	-0.0185
Decision Tree	0.8145	0.7592	-0.0553	0.8162	0.8060	-0.0102
Logistic Regression	0.8419	0.7885	-0.0534	0.8622	0.8384	-0.0238
Naive Bayes	0.7733	0.7288	-0.0445	0.7313	0.7502	0.0189
CNN 1D	0.7235	0.7308	0.0073	0.7293	0.7469	0.0176

FIGURE 10.8 – Tableau récapitulatif des F1-scores avant et après augmentation TVAE pour les six classifieurs.

Sur `df_selected`, le TVAE dégrade souvent les performances du Decision Tree et du KNN en ajoutant des régions de décision complexes. À l'inverse, SVM et Régression Logistique restent stables, profitant de leur robustesse.

Sur `df_transformed`, l'impact est nettement plus favorable. La transformation `log1p` combinée au z-score permet au TVAE de générer des échantillons de bien meilleure qualité statistique, confirmant l'importance cruciale du prétraitement.

Enfin, le CNN 1D se distingue par sa capacité à modéliser les interactions non linéaires ; il tire particulièrement profit des données synthétisées après transformation par rapport aux classifieurs classiques.

Conclusion générale

Bilan du pipeline

Ce projet a mis en œuvre un pipeline complet de détection de ransomwares, couvrant l'ensemble des étapes depuis la préparation des données brutes jusqu'à l'évaluation comparative de modèles de classification sur données réelles et synthétiques. Le tableau ci-dessous résume les transformations successives appliquées au dataset initial.

Table 10.1 – Récapitulatif du pipeline global.

Étape	Phase	Dimension	Résultat clé
1	Nettoyage (doublons + outliers)	588×64	218 doublons supprimés ; 11,26 % des valeurs écrêtées (IQR $\times$ 3).
2	Feature Engineering	588×96	32 features construites : agrégations, ratios, indicateurs binaires, diversité.
3	Sélection (vote majoritaire)	588×30	30 features discriminantes retenues sur 96 par consensus Mann-Whitney, information mutuelle et Random Forest.
4	Transformation (log1p + z-score)	588×30	Distributions normalisées ; séparabilité améliorée entre classes.
5	Augmentation TGAN	$1\,088 \times 30$	500 échantillons synthétiques générés par GAN séquentiel (LSTM).
6	Augmentation CTGAN	$1\,088 \times 30$	500 échantillons synthétiques générés par GAN conditionnel.
7	Augmentation TVAE	$1\,088 \times 30$	500 échantillons synthétiques générés par encodeur variationnel.

Comportement des générateurs synthétiques

Les trois modèles de génération évalués — TGAN, CTGAN et TVAE — présentent des profils complémentaires face aux données de détection de ransomwares.

Table 10.2 – Comparaison des générateurs synthétiques.

Critère	**TGAN**	**CTGAN**	**TVAE**
Paradigme	GAN séquentiel LSTM (adversarial)	GAN conditionnel par classe (adversarial)	Encodeur–décodeur variationnel (reconstruction)
Stabilité d'entraînement	Moyenne (convergence plus rapide, variance plus élevée)	Moyenne (instabilité adversariale possible)	Élevée (pas de mode collapse)
Fidélité marginale	Dépend de la qualité des LSTM	Meilleure sur features multimodales (*mode-specific normalization*)	Bonne sur features unimodales
Impact sur `df_selected`	Variable (DT et KNN instables)	Homogène (moins de dégradations extrêmes)	Contrasté (DT et KNN sensibles)
Impact sur `df_transformed`	Positif ; CNN 1D aligné avec la logique séquentielle	Positif ; CNN 1D en bénéficie particulièrement	Globalement positif après log1p + z-score

Comportement des classifieurs

L'évaluation des six classifieurs sur les trois datasets augmentés permet de dégager des tendances claires quant à leur sensibilité à la génération synthétique.

— **SVM (RBF) :** classifieur le plus robuste à l'augmentation synthétique quel que soit le générateur utilisé. Sa construction par maximisation de marge le protège des variations de densité locale introduites par les données synthétiques.

— **Régression Logistique :** performances stables avant et après augmentation sur les deux datasets. Son hypothèse de séparabilité linéaire est bien satisfaite après la transformation log1p + z-score, ce qui lui confère une grande indépendance vis-à-vis du générateur.

— **CNN 1D :** seul modèle de Deep Learning de la comparaison ; il tire le meilleur

parti de l'augmentation sur `df_transformed`, où sa capacité à exploiter des interactions locales entre features est renforcée par la cohérence statistique des données synthétiques.

— **Naïve Bayes :** bénéficie principalement de l'augmentation sur `df_transformed`, où les distributions gaussiennes supposées par le modèle sont mieux vérifiées après normalisation.

— **Decision Tree et KNN :** classifieurs les plus sensibles à la qualité des données synthétiques. Leur dépendance à la topologie locale les expose aux artefacts de génération ; les gains observés sont moins constants et plus dépendants du générateur utilisé.

Enseignements généraux

Enseignements clés du projet

1. **Le prétraitement conditionne la qualité de la génération :** les données synthétiques produites sur `df_transformed` (log1p + z-score) sont systématiquement de meilleure qualité que celles produites sur `df_selected`, quel que soit le générateur. La normalisation préalable est une condition nécessaire à l'efficacité de l'augmentation synthétique.
2. **Le Feature Engineering améliore la discriminabilité :** les 32 features construites (ratios, agrégations, indicateurs binaires) ont été validées par trois méthodes indépendantes et ont contribué à réduire la dimensionnalité de 96 à 30 variables sans perte d'information discriminante.
3. **Aucun générateur n'est universellement supérieur :** TGAN présente la convergence la plus rapide ; CTGAN produit les distributions marginales les plus fidèles ; TVAE offre la stabilité la plus élevée. Le choix du générateur doit être guidé par les contraintes de temps d'entraînement et la nature des classifieurs cibles.
4. **Les modèles linéaires et à marge sont les plus fiables pour la détection de ransomwares :** SVM et Régression Logistique maintiennent des performances élevées et stables, indépendamment de l'augmentation. Le CNN 1D représente l'alternative Deep Learning la plus prometteuse sur données transformées.

Ressources et Code Source

L'intégralité du code source, allant du prétraitement des données à l'évaluation des modèles, est disponible en accès libre sur GitHub. Vous y retrouverez également les notebooks et les implémentations détaillées :

Repository GitHub :
ilyasbejja/ransomware_detection